

Sharif University of Technology
Department of Mechanical Engineering

Path Planning and Obstacle Collision Avoidance

Robotics Course – Homework 6

Haniyeh Zahra Budaqi – 99105956
Instructor: Dr. Behzadipour

First Term, 1402 (Fall 2023)

Contents

1	Problem Statement	2
1.1	Question 1	2
1.2	Question 2	2
1.3	Question 3	2
1.4	Question 4 (Bonus)	2
2	Answers to the Questions	3
2.1	Question 1	3
2.2	Question 2	5
2.3	Question 3	6
2.4	Question 4	8

1 Problem Statement

In this problem we are asked to use the potential field method to design the motion path of the end-effector of a robot in a two-dimensional plane containing predetermined obstacles.

1.1 Question 1

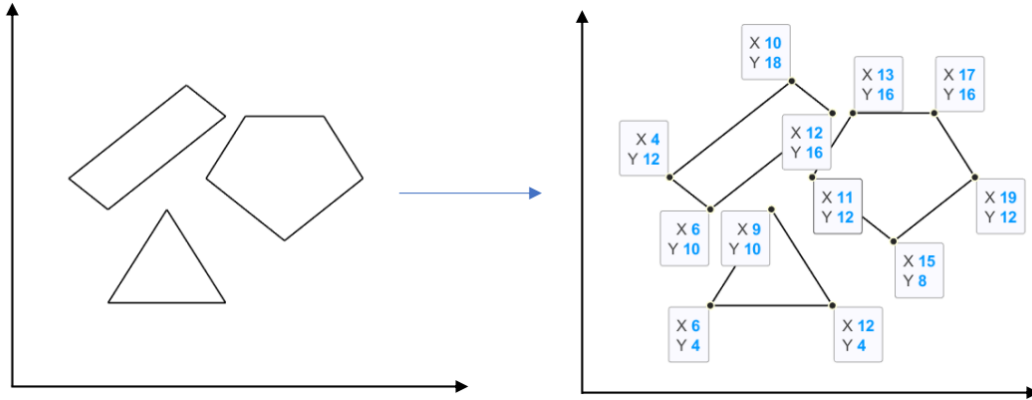
In the first part of the problem, a function must be written in MATLAB that generates the points of the path. The inputs of this function are the starting and ending points of the motion, η as the scaling coefficient of the attractive field, and matrix B . Matrix B contains the data related to the obstacles. The function keeps generating path points until the last point generated by the function comes within a distance of 0.1 of the ending point. The output of this function is matrix P , which contains the coordinate data of the path points.

```
function P = Path_generator (Xs, Xf, eta, B)
```

1.2 Question 2

In this part, an example is given so that we can test our function against it. We take $\eta = \alpha = 1$, and the value of the variable epsilon (which determines the step size of the path) is also equal to 0.1. The coordinates of the obstacle vertices are given in the figure below. The coordinates of the start and end points are as follows:

$$X_s = (1, 10), \quad X_f = (22, 12)$$



1.3 Question 3

We add a new obstacle in the shape of a line segment between the two points (17.5, 10) and (17.5, 8). We must now test the function again. The question asks where the path gets stuck.

1.4 Question 4 (Bonus)

In this part we are asked to implement a Random Walk algorithm and use it to solve the problem encountered in Part 3 of the question. This algorithm comes into play whenever the Potential Field Algorithm (PFA) gets stuck at a local-minimum point. Then, using Random Walk, we take a jump with a step mechanism of size a ; in this problem we set a equal to 6. The jump angle is random, as

long as we are certain that no collision with an obstacle will occur. Another method could also be proposed for determining the optimal angle for escaping the local-minimum point. Then, the point obtained by this method is fed back into the potential field algorithm, and it is checked whether the path can be continued to the end point using it or not.

2 Answers to the Questions

2.1 Question 1

We define matrix B in MATLAB as a variable of type $\text{Cell}_{2 \times n}$, such that each column of it contains the data of one obstacle, from obstacle 1 to obstacle n . The element stored in $B\{1,i\}$ represents the number of sides of obstacle i , and the element $B\{2,i\}$ is a matrix that holds the coordinates of the obstacle's vertices, in the order in which they connect to one another, one after another.

We implement the potential field algorithm in MATLAB in the manner explained in the course notes. The final code is as follows:

```
function P = Path_generator(Xs, Xf, eta, B)

%Xs is the starting point
%Xf is the finishing point
%eta is our chosen scaling factor
%B is the cell in which we have stored obstacle data

k = 1; %just a variable to keep track of the number of the points in the
      path
X = []; %initiating the path points matrix
X(:,k) = Xs; %starting point
[~,n] = size(B); %n: the number of the obstacles

while norm(Xf-X(:,k)) > 0.1 %the threshold we chose to get to Xf
    x = X(:,k); %making things a shade simpler
    F_att = -eta*(x-Xf); %attraction force calculated
    F_rep = zeros(2,n);

    for i = 1:n %we're gonna find Bi for n objects
        m = B{1,i}; %m: number of the points of the object i-th
        points = B{2,i}; %coordinates of those points
        B_dis = ones(2,m);
        for j = 1:m
            p1 = points(:,j);
            if j == m
                p2 = points(:,1);
            else
                p2 = points(:,j+1);
            end

            if m == 1
                p2 = points(:,2);
            end

            side = p2 - p1; %one side of the polygon
            rot_mat = [0 -1; 1 0]; %rotation matrix (for 90 degrees)
            D = [side, rot_mat*side];
```

```

        coeff = D\(x - p1); %coeff consists of alpha and beta
        alpha = coeff(1); %we just needed one of them, so we choose
            alpha

        if alpha > 0 && alpha < 1 %finding where Bi of this side lies
            on
                Bf = alpha*(p2 - p1) + p1;
        elseif alpha <= 0
            Bf = p1;
        else
            Bf = p2;
        end

        B_dis(:,j) = Bf; %Bi for one side found, got called Bf, and
            saved
    end

    Bi = B_dis(:,1); %comparing all Bi's to find the closest one to x
    if m > 1
        for j = 2:m
            if norm(x - B_dis(:,j)) < norm(x - Bi)
                Bi = B_dis(:,j);
            end
        end
    end
    % Bi found

    ro_i = norm(Bi - x); %calculating the repulsive forces
    alpha_i = 1;
    ro_o_i = 2; %alpha_i and ro_o_i are the same for all obstacles
    if ro_i <= ro_o_i
        F_rep(:,i) = alpha_i*(1/ro_i^3)*((1/ro_i) - (1/ro_o_i))*(x-Bi);
    end
end

sumFrep = zeros(2,1); %summing up the repulsive forces now
for t = 1:n
    sumFrep(1) = sumFrep(1) + F_rep(1,t);
    sumFrep(2) = sumFrep(2) + F_rep(2,t);
end

F = F_att + sumFrep; %total force be like:
Fd = F / norm(F); %here comes the unit vector of the path we pave
eps = 0.1; %and here is the step
X(:, k+1) = x + eps*Fd; %next point of the path
k = k+1; %let's do it all over

end

X(:,k) = Xf; %adding our last point to the path
P = X; %path generated

end

```

2.2 Question 2

We define cell B and, following the definition we used in Part 1, place the problem's data into it. Then, we define the start and end points of the path and assign a value to the variable η .

After that, using the function we wrote in Part 1, we design the path and obtain the coordinates of the path points as matrix P .

To draw the obstacles, we use the `polyshape` function, storing each obstacle as a polygon object according to its coordinates, and then we draw them using the `plot` command.

Finally, we store the X and Y coordinates of the path separately and plot them on our figure as well.

The complete final script for this part of the question, along with the resulting image of the obstacles and the designed path, is as follows:

```
clc; clear;
Obst = [3,4,5]; %this vectors contains the number of sides for each
            obstacle
n = length(Obst); % n = number of the obstacles

% saving obstacles data in a cell
B = cell(2,n);
B{1,1} = Obst(1); B{2,1} = [9 6 12; 10 4 4];
B{1,2} = Obst(2); B{2,2} = [4 10 12 6; 12 18 16 10];
B{1,3} = Obst(3); B{2,3} = [11 13 17 19 15; 12 16 16 12 8];

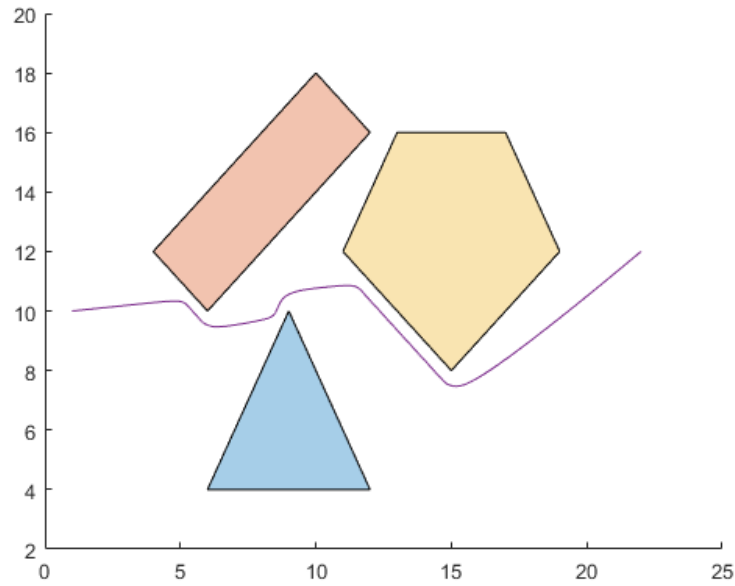
Xs = [1;10];
Xf = [22;12];
eta = 1;

P = Path_generator(Xs,Xf,eta,B);

% plotting obstacles
obs1 = polyshape(B{2,1}(1,:), B{2,1}(2,:));
obs2 = polyshape(B{2,2}(1,:), B{2,2}(2,:));
obs3 = polyshape(B{2,3}(1,:), B{2,3}(2,:));

plot(obs1);
hold on
plot(obs2);
plot(obs3);

path_x = P(1,:); path_y = P(2,:);
plot(path_x, path_y)
```



2.3 Question 3

In a new script, we again define cell B , and this time we also place the coordinates of the given points into it. In fact, we define an obstacle with a single side (edge). We then feed the data into the path-generating function again, and observe that the function enters the `while` loop and never exits it.

The script written is:

```
Obst2 = [3,4,5,1]; %this vectors contains the number of sides for each
    obstacle
n2 = length(Obst2); % n = number of the obstacles

% saving obstacles data in a cell
B2 = cell(2,n2);
B2{1,1} = Obst2(1); B2{2,1} = [9 6 12; 10 4 4];
B2{1,2} = Obst2(2); B2{2,2} = [4 10 12 6; 12 18 16 10];
B2{1,3} = Obst2(3); B2{2,3} = [11 13 17 19 15; 12 16 16 12 8];
B2{1,4} = Obst2(4); B2{2,4} = [17.5 17.5;10 8];

Xs = [1;10];
Xf = [22;12];
eta = 1;

P = Path_generator(Xs,Xf,eta,B2);

obs2_1 = polyshape(B2{2,1}(1,:), B2{2,1}(2,:));
obs2_2 = polyshape(B2{2,2}(1,:), B2{2,2}(2,:));
obs2_3 = polyshape(B2{2,3}(1,:), B2{2,3}(2,:));
obs2_4_x = B2{2,4}(1,:); obs2_4_y = B2{2,4}(2,:);

plot(obs2_1);
hold on
```

```

plot(obs2_2);
plot(obs2_3);
plot(obs2_4_x, obs2_4_y);

path_x = P(1,:); path_y = P(2,:);
plot(path_x, path_y)

```

We plot the path generated up to this point by issuing a command from the debug section:

```

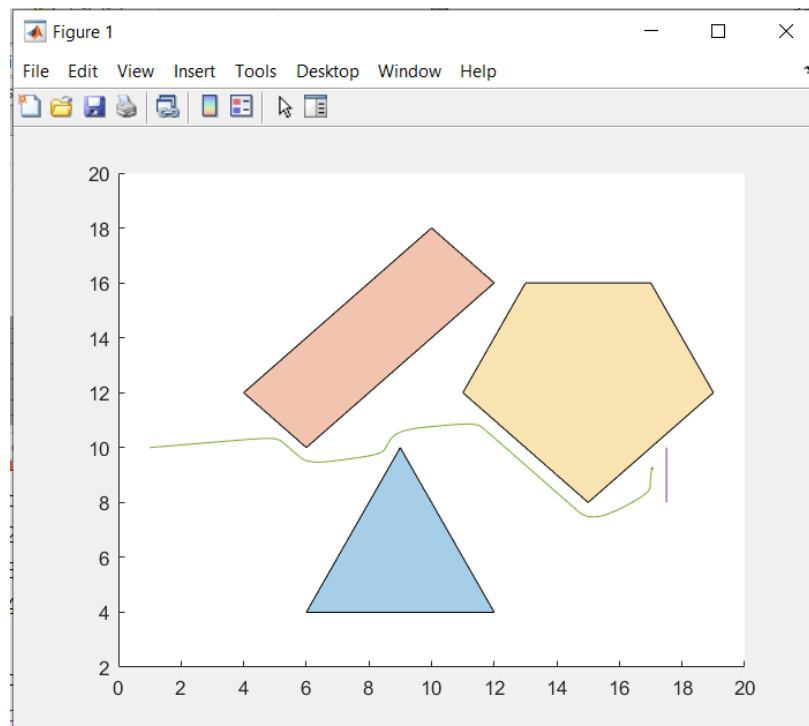
K>> obs2_1 = polyshape(B{2,1}(1,:), B{2,1}(2,:));
obs2_2 = polyshape(B{2,2}(1,:), B{2,2}(2,:));
obs2_3 = polyshape(B{2,3}(1,:), B{2,3}(2,:));
obs2_4_x = B{2,4}(1,:); obs2_4_y = B{2,4}(2,:);

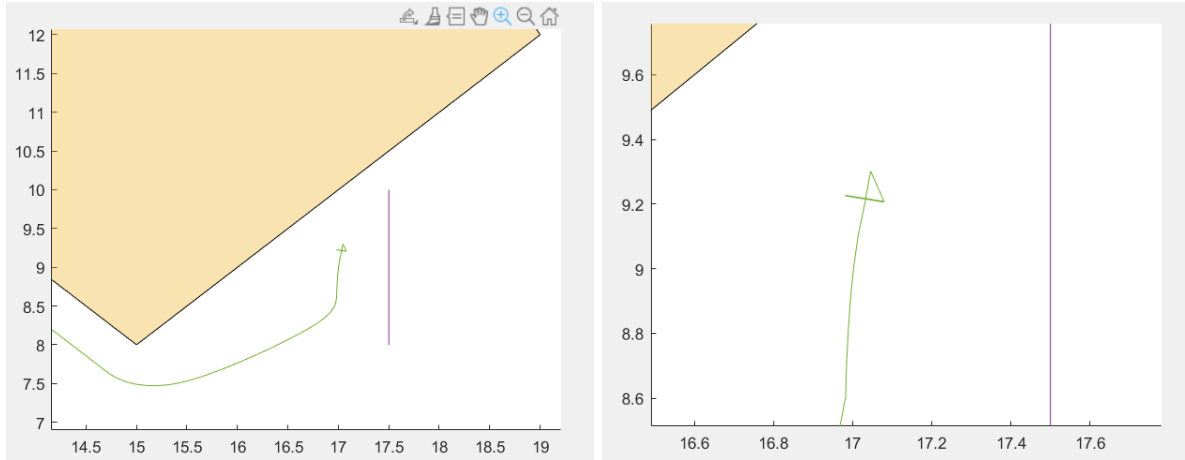
plot(obs2_1);
hold on
plot(obs2_2);
plot(obs2_3);
plot(obs2_4_x, obs2_4_y);

path_x = X(1,:); path_y = X(2,:);
plot(path_x, path_y)

```

We obtain the following plot:





It appears that the path has become stuck at a local minimum. However, considering that a break (kink) occurs in the path, it can be said that the path has become trapped between two obstacles, with no way forward or back, and it has fallen into an infinite back-and-forth loop.

2.4 Question 4

*The codes and models are included in the appendix.